WHITEPAPER · v1.3 · JUNE 2026

RALPH

A Bittensor Subnet for Decentralized, Autonomous AI Research

An attested, lineage-bound recipe substrate for small-scale pretraining research — and the public diff corpus it produces — run by an autonomous research network on Bittensor.

In one line. Ralph is a cryptographically-provenanced tree of training-recipe improvements, each one bound to a private eval pool that grows with the tree and cannot be forked — running live on Bittensor, and licensed to applied-research teams as ralph-diffs, the dataset of what worked, what didn't, and by how much.

Important Notice

This document is a draft for discussion. It is provided for informational purposes only and does not constitute investment advice, a solicitation, or an offer to sell or buy any token, security, or financial instrument. Forward-looking statements about technology, markets, roadmaps, and token mechanics are estimates subject to material change. Decentralized-AI tokens — including Bittensor subnet "alpha" tokens — are volatile, experimental, and may be subject to evolving regulatory treatment in many jurisdictions. Nothing here should be relied upon for financial, legal, or tax decisions; readers should consult their own qualified advisors. All third-party data is attributed to its source and was current as of the dates cited.

Contents

1. Executive Summary

Ralph is a continuously improving, open-source training recipe and the public knowledge corpus behind it — both produced by a decentralized network of autonomous research agents competing on Bittensor. It is closer to a permanent scientific instrument than a software company: the network exists to produce research artifacts the world keeps and uses forever; the subnet and its token are the funding and coordination mechanism, not the deliverable.

1.1 What Ralph is

Ralph is one thing, not four. It is a **substrate**: a single, continuously growing tree of training-recipe improvements, where every node is an attested run, every edge is a measured improvement, and the tree is bound to a private evaluation pool — the *sealed-shard accumulator*, now rolling out (Section 5) — designed so a fork can take the public recipe but not the private evidence underneath it. A frontier lab can copy the public recipe at the tip of the tree in an afternoon. What it cannot copy is the accumulated sealed, attested record beneath each step — which changes helped, which hurt, and by exactly how much, under conditions that were fixed in advance and proven on hardware.

That substrate produces three things the world keeps:

1. **A canonical, open training recipe.** A public Git repository holding the best-known open recipe for small-LLM pretraining — optimizer schedule, data mixture, architecture, initialization — at the current head of the lineage. Anyone can clone it and train with state-of-the-art settings. It improves whenever the network crowns a new king.
2. **ralph-diffs — the diff corpus, and the product.** Every change the network has evaluated, published as a structured dataset: the recipe diff, its measured effect across the evaluation ladder, its multi-seed variance, the hardware-attestation hash, and a pointer to the parent it built on. This is the artifact frontier labs generate internally and never release — the training signal a recipe-proposing model learns from. The recipe itself (1) is a free public good; ralph-diffs is published under a share-respecting open data license, and offered to applied-research teams as a commercial ingestion license for training on it directly.
3. **A model lineage — Ralph-1, Ralph-2, ...** Periodic open-weights reference models trained on the recipe at a moment in time. They are not the headline; they are the receipt: proof that the recipe works and that the improvement compounds.

The live network of miners, validators, and the token that pays them is the **engine**, not the deliverable. People do not buy a factory; they use what it makes. The right frame is closer to a permanent scientific instrument than a software company — and the instrument is precisely the attested lineage plus the sealed pool it is measured against.

Ralph launches with a single lineage at the ~124M-parameter ladder scale — the same scale as the public NanoGPT-Speedrun work where automated research is most active today. It is live on Bittensor mainnet, netuid 40.

1.2 Who gets what

A short map of who interacts with Ralph, and what they receive.

Audience	What they get
ML researchers and small labs	A clone-able, frontier-tracking training recipe that saves months of trial-and-error — plus a searchable database to answer "has anyone tried this?"
Startups and companies	The recipe and corpus, plus the option to post a paid bounty asking the network to improve a specific training metric on their use case and data.
Bittensor stakers and dTAO holders	Alpha exposure to a subnet whose underlying assets — the recipe and the corpus — compound in real value, with revenue-funded buybacks driving non-speculative buy pressure.
Miners and contributors	Paid emissions for running autonomous research agents, with every accepted finding becoming a permanent, attributed entry in the public record under their name.
The AI field at large	A citable, openly-licensed corpus of training experiments — including negative results — that makes the whole field less wasteful and easier for outsiders to enter.

1.3 How to use Ralph, in practice

A network of public artifacts is only as useful as the doors that lead into it. Three short examples of who walks through which door:

The ML researcher with a specific problem. A researcher who wants, say, a better 70B-class coding model starts at the experiment-record corpus and queries across tracks for techniques validated under similar conditions — model scale, modality, objective. They clone the canonical recipe of the closest matching track (LLM coding post-training, if it exists), inheriting validated data mixtures and post-training schedules they would otherwise spend months rediscovering. If their problem is too specific to any existing track, they post a bounty for it (next paragraph). Either way they start far ahead of scratch, with reproducible evidence behind every choice they inherit.

The startup with a metric and a budget. A company with a training pipeline and a target metric — lower perplexity on their domain, faster inference on their checkpoint, better recall on their retrieval setup — posts a bounty: setup, eval, time horizon, reward pool. Miners compete to improve that exact metric on the company's own setup; validators verify each candidate in the same staged sandbox; the company pays only for verified improvement. They never run a training cluster or hire a research team for the problem.

The contributing miner. A small operator with one or a few GPUs picks a track that fits their hardware — small LLM pretraining is approachable on a single H100 — configures an autonomous research agent through a *program.md* file, and runs it continuously. They submit any improvement they find as a patch; every verified contribution earns emissions and a permanent attributed entry in the experiment-record corpus under their name. They are not paid for compute; they are paid for *discoveries*.

1.4 Why now

Artificial-intelligence progress is colliding with two hard limits at the same time. The first is **cost**: a single frontier training run has grown from roughly $4.6M for GPT-3 in 2020 to an estimated $40–100M+ for GPT-4, with the amortized hardware cost of frontier runs rising about **2.4× per year** since 2016 and credible projections placing the largest 2027 runs above **$1 billion**. The second is the **human researcher**: the slow, serial, expensive element that decides which experiments to run and what the results mean.

R&D staff account for an estimated 29–49% of frontier model development cost, and the experiments and ablations around a successful run consume an additional 10–30% of its compute on their own.

In March 2026, Andrej Karpathy released **autoresearch** — a deliberately tiny project in which an AI agent is given a real single-GPU model-training setup, a metric, and a fixed time budget, and left to experiment overnight. It edits code, trains, checks whether the result improved, keeps or discards, and repeats. Karpathy, a researcher with two decades of experience who believed his baseline was already well-tuned, reported that the agent found improvements he had missed. The repository drew more than 80,000 GitHub stars within weeks. Shopify engineers then generalized the same loop beyond model training, using it to optimize 40+ internal engineering metrics and cutting one continuous-integration pipeline's build time by 65%.

Autoresearch proved a thesis. But as shipped it is a **toy** by design: one GPU, one file, one metric, one loop, no defenses against an agent that games its own benchmark. Ralph is the proposal to evolve that toy into a serious, decentralized research engine — and to host it on Bittensor, the only production network that already pays, in a live market, for verifiable contributions of machine intelligence.

> **The core insight.** Self-improvement only works reliably where outcomes are objectively verifiable — code compiles or it does not; a model's benchmark score goes up or it does not. This "verifiability constraint" is exactly the property Bittensor's validator model is built to exploit. Autonomous research over a measurable training metric is therefore a near-perfect fit for a subnet: the thing miners optimize is the thing validators can independently and cheaply check.
>
> **The whitespace.** Bittensor hosts 128 live subnets for inference, distributed training execution, storage, and prediction — but none yet runs autonomous research as a competitive market. Ralph is designed to occupy that gap.

How it works, in one paragraph. Miners operate autonomous "research organizations" — agent swarms configured by editable *program.md* files — that submit improvements as auditable patches against the canonical training repository. They run their own training stages: a cheap proxy run filters obviously bad ideas; promising patches are confirmed across multiple seeds; survivors are tested at larger model scale. Validators do not retrain in the hot path. They verify each submission cheaply against hidden, rotating evaluation sets, and a small, randomly-selected fraction of submissions are subjected to a full re-train audit that catches fraud. Scoring rewards **reliable improvement per compute dollar** — not raw GPU budget — normalizing quality gains against compute cost, stability, and complexity. Patches that survive are merged into the canonical recipe, which every future miner builds upon. Each accepted contribution permanently raises the floor for the entire network. Sections 5 and 6 detail the mechanism, the token economics, and the buyback program that connects revenue back to the subnet.

Why it matters. Automated AI research is, by the explicit admission of the largest labs, the most valuable direction in the field — OpenAI has called it the company's "North Star." Today that capability is being built privately, behind closed doors, by a handful of organizations that can afford nine- and ten-figure compute budgets. Ralph's wager is that the research loop itself — not just the trained weights — can be made an open, permissionless, community-owned public good, funded by a token that rewards verifiable scientific contribution rather than speculation alone.

2. The Problem: AI Research Has Hit a Cost Wall

2.1 The exploding cost of frontier training

The cost of training a state-of-the-art model has risen faster than almost any input in modern computing. Epoch AI, which maintains the most rigorous public database of notable models, finds that the amortized hardware-plus-energy cost of a frontier final training run has grown roughly **2.4× every year since 2016**, while training compute itself has scaled 4–5× per year. The headline figures depend heavily on accounting method — amortized hardware versus cloud-rental pricing versus full program cost — which is why credible sources disagree by an order of magnitude. The trajectory, however, is not in dispute.

Estimated training cost of selected frontier models. Figures vary by methodology; ranges reflect the spread across Epoch AI, the Stanford AI Index, and public reporting [1–4].

Model	Year	Estimated training cost	Notes
GPT-3	2020	~$4.6M	~3.1e23 FLOP
GPT-4	2023	$40M–$100M+	Epoch ~$40M amortized; Stanford ~$78M cloud-rental; >$100M total per OpenAI
Gemini Ultra	2024	$30M–$192M	~35 MW power draw
Llama 3.1 405B	2024	~$170M	Open-weight; trained on H100 clusters
DeepSeek V3	2024	~$5.6M	Compute only; excludes experiments, infra, failed runs
Frontier class	2026	$200M–$500M	GPT-5 / Gemini Ultra-class runs, 1e26–1e27 FLOP
Frontier class	2027 (proj.)	**$1B–$3B**	*Power capacity, not silicon, becomes the binding constraint*

The DeepSeek V3 line is instructive. Its widely-quoted $5.6M figure is real but narrow: it covers the final run's compute and, as multiple outlets noted, **excludes the experimentation, infrastructure, and failed runs** that surround it. That exclusion is precisely the part of the cost structure this whitepaper is about.

2.2 The hidden tax: experiments, ablations, and human researchers

A frontier model is not produced by one training run. It is produced by hundreds or thousands of smaller experiments — architecture variants, optimizer sweeps, data-mixture trials, learning-rate schedules — most of which fail. Epoch AI estimates that total compute across a model's development is **1.2–4× the cost of the final run alone**, and that failed runs and ablations typically burn **10–30% of a successful frontier run's FLOP** before that run is even launched.

Then there is labor. Epoch's in-depth cost decomposition of GPT-3, OPT-175B, GPT-4, and Gemini Ultra finds that **R&D staff account for 29–49% of total amortized development cost** — a share comparable to the hardware itself. Senior researchers at leading labs command total compensation well north of $500,000, and the most sought-after talent reportedly commands $1–5M. The expensive, scarce resource is not only the GPU. It is the human deciding what to do with it.

This is the cost the industry has not yet automated. Hardware efficiency improves with every chip generation. Inference cost has fallen dramatically — the price of GPT-3.5-level quality dropped roughly 280× in 18 months. But the cost of *running the search* — of exploring the space of possible models — still scales with human attention and with compute spent on dead ends.

2.3 The researcher-in-the-loop bottleneck

In the podcast accompanying autoresearch's release, Karpathy framed the problem in terms of leverage. The constraint, he argued, is no longer typing speed or even ideas — it is that a human must *be present* to prompt the next step, read the next result, and decide what comes after. "You need to take yourself outside," he said; "you have to arrange things such that they're completely autonomous." He went further about research specifically: experienced researchers, he suggested, often have "way too much confidence … they shouldn't actually be enacting" ideas by hand. The role of the human collapses to maintaining a queue of ideas and occasionally reviewing a feature branch; autonomous workers pull from the queue, try things, and merge what works.

This is the bottleneck Ralph attacks. Not the GPU. The serial, expensive, ego-laden, sleep-requiring researcher-in-the-loop — and the fact that, today, the few organizations capable of automating that loop are doing so privately and keeping the result.

> **The problem in one sentence.** AI research is gated by a cost structure — failed experiments, idle researchers, nine-figure budgets — that a small number of well-capitalized labs are now racing to automate behind closed doors, while everyone else is locked out of both the process and its rewards.

3. The Breakthrough: Autoresearch

3.1 What Karpathy's autoresearch demonstrated

Released in March 2026, **autoresearch** (github.com/karpathy/autoresearch) is, in the author's framing, the origin story of a future in which "research is now entirely the domain of autonomous swarms of AI agents." Mechanically it is small: a simplified single-GPU implementation of nanochat training, with three files that matter — *prepare.py* (fixed data prep, not modified), *train.py* (the model, optimizer and training loop, edited freely by the agent), and *program.md* (Markdown instructions that configure the autonomous research "org," edited by the human).

The design choices are the interesting part. Training runs for a **fixed 5-minute wall-clock budget**, which makes every experiment directly comparable regardless of what the agent changed — roughly 12 experiments per hour, about 100 overnight. The metric is *val_bpb* (validation bits per byte): lower is better, and vocabulary-independent, so architectural changes are scored fairly. The agent edits one file, runs, checks the metric, keeps or reverts, repeats. The human never touches Python.

The result that mattered was not the speedup; it was the **epistemic** one. Karpathy — by his own account a researcher with two decades of experience who had hand-tuned the baseline and believed it was already in good shape — let the loop run overnight and it returned tunings he had not seen, including a missed weight-decay term on value embeddings and inadequately tuned optimizer betas that interact with other hyperparameters. An autonomous loop out-searched an expert on the expert's own problem. The repository passed 80,000 stars and 11,000 forks within weeks of release.

3.2 Generalization: the Shopify case

If autoresearch were only about training nanochat models, it would be a curiosity. Its significance is that the loop generalizes to **any process with a measurable metric**. In April 2026, Shopify engineers published an account of doing exactly that. A developer frustrated by 30-minute CI failures built a thin extension of autoresearch pointed not at *val_bpb* but at build time. The loop formed hypotheses, tested them, kept what ran faster, discarded crashes and regressions — and cut the Polaris component pipeline's build time by **65%**. The project, *pi-autoresearch*, was open-sourced, drew thousands of GitHub stars, and spread internally into an "#autoresearch-wins" channel where teams reported unit tests running hundreds of times faster, components mounting 20% faster, and more — 40+ metrics improved across the company.

The Shopify author named the deeper point precisely: autoresearch "does work nobody would attempt manually. No one's sprint plan includes 'spend three months reducing build time by 30%.'" An autonomous agent has no competing priorities, does not get bored, and has no deadline pulling it elsewhere. The toil humans correctly deprioritize is the ideal workload for an autonomous loop.

3.3 The limits of autoresearch today

Autoresearch is, by explicit design, a baseline. Its own documentation calls *program.md* a "crappy attempt" at describing how an auto-researcher should work, and invites iteration. The gap between that baseline and a serious research engine is exactly the design space Ralph occupies:

- **One GPU, one loop.** Autoresearch is single-GPU and single-agent. Real research happens across many parallel lines of inquiry and, eventually, across distributed clusters.
- **One short run, one metric.** A 5-minute run on a tiny model is a fast filter, not proof. Many tricks that help at small scale fail at large scale; a single run is statistically noisy; *val_bpb* alone ignores throughput, memory, stability, and downstream benchmark performance.
- **No defense against gaming.** If an agent can see the validation data — or edit the evaluation or scoring code — it will, sooner or later, optimize the measurement instead of the model. Autoresearch has no sandbox, no hidden evals, and no diff scanner because it does not need them: it is a trusted, single-user tool.
- **Results are not comparable across people.** Because the time budget is platform-relative, two researchers running autoresearch cannot compare results. There is no shared, canonical baseline that accumulates everyone's improvements.
- **No incentive layer.** Autoresearch rewards its single user with a better model. It has no mechanism to coordinate — or pay — thousands of independent contributors competing to improve a common artifact.

Each limitation is a design requirement for Ralph. Section 5 turns them into an architecture; the rest of this section establishes *why now* is the moment to build it.

4. Market Analysis: Why Now

4.1 The automated-research land grab

Ralph is not betting that automated AI research will become important. The largest labs have already made that bet, publicly, with their balance sheets. According to reporting in early 2026, OpenAI has declared an autonomous researcher its **"North Star"** direction — targeting an "autonomous research intern" able to take on small research problems by itself, with a fully automated multi-agent research system to follow. Reporting has described internal projections involving research agents priced on the order of $20,000 per month and revenue ambitions in the hundreds of billions by the end of the decade. Whatever the precision of any single figure, the strategic signal is unambiguous: the frontier labs believe the highest-value product they can build is a system that does research.

The research literature has been converging on the same point from the other direction. Sakana AI's *The AI Scientist* generates end-to-end research papers for roughly $15 each; its successor produced what was reported as the first fully AI-generated, peer-review-accepted workshop paper. A growing field of "AI scientist" and automated-ML-research systems — from AutoML lineage through MLAgentBench, ShinkaEvolve, and AutoSOTA — is steadily widening the slice of the research pipeline that agents can execute without a human. Karpathy's framing of frontier labs as already "trying to recursively self-improve, roughly speaking" is the consensus, not a fringe view.

The signal is no longer confined to labs and research papers — it has reached mainstream developer adoption. In February 2026, Nous Research released **Hermes Agent**, an open-source agent built around a closed-loop self-evolution mechanism: it extracts and refines its own skills from use, and applies evolutionary optimization to its prompts and code against benchmarks. Within roughly three months it passed 110,000 GitHub stars, and in May 2026 it briefly overtook the leading session-based agent on daily usage rankings [18]. The implication for Ralph is confirmatory rather than competitive — Hermes is a single-user personal agent, not a research network — but the signal is unambiguous: "a system that improves itself" has become one of the fastest-adopted ideas in open-source AI. Ralph applies the same instinct one layer deeper, and in the one setting where self-improvement can genuinely be trusted: a verified, incentivized market improving a shared artifact rather than a private one.

4.2 The verifiability constraint — and why it is decisive

Self-improving systems have one well-documented failure mode and one well-documented success condition. Analyses of self-improving agents in 2026 converge on what is sometimes called the **verifiability constraint**: autonomous self-improvement works reliably *only* in domains where outcomes are objectively verifiable. Code compiles or it does not. An algorithm runs faster or it does not. A model's loss or benchmark score moves in a direction that can be measured without human judgment.

This is the single most important reason to believe a research subnet can work where a vaguer one would not. Bittensor's entire architecture is a machine for **rewarding verifiable contributions of intelligence**: validators must be able to independently and cheaply check the value of what miners produce. Subjective tasks make that hard. Model-training research does not: *val_bpb*, held-out benchmark accuracy, tokens-per-second throughput, peak memory, and divergence rate are all numbers a validator can reproduce in a sandbox. The thing Ralph asks miners to optimize is, by construction, the thing it can verify. Autoresearch and Bittensor were, in effect, designed for each other.

> **Why this is the right task for a subnet.** A research subnet inherits autoresearch's objective metric and Bittensor's objective verification. Miners cannot win by being persuasive, only by being measurably right — and validators can prove it by re-evaluating the resulting checkpoint against a hidden, rotating eval set, with a probabilistic audit re-running submissions in full to catch fraud, and a continuous-attestation chain produced by the official proof-test container that records every epoch of the run for asynchronous validator verification. The incentive and the verification point at the same target.

4.3 The rise of decentralized training

A research subnet would be a fantasy if training could only happen inside hyperscale datacenters. Until recently that was the assumption. It no longer holds. Epoch AI reports that the computational scale of decentralized training projects has grown roughly **600,000× since 2020** — an implied ~20× per year, faster even than frontier training's 5×. Concrete results have followed:

- **Prime Intellect** trained INTELLECT-1 across globally distributed, low-bandwidth hardware, then shipped production models at 10B and 32B parameters and an open 106B-parameter mixture-of-experts model, INTELLECT-3.
- **Nous Research's Psyche** and **Pluralis** have built distributed-training networks and dashboards demonstrating training across heterogeneous, internet-connected nodes.
- **Templar**, a Bittensor subnet, trained a 72B-parameter model ("Covenant-72B") permissionlessly across the network — by one account on 1.1 trillion training tokens contributed by 70+ participants on commodity hardware, reaching an MMLU score competitive with a prior-generation 70B model.

Decentralized training is still smaller than centralized frontier compute — by Epoch's estimate the largest decentralized network runs roughly 300× below a frontier datacenter's throughput. But the relevant fact for Ralph is different: **research does not require frontier scale**. As Karpathy emphasized, the productive pattern is to do the exploration on small models and extrapolate. Small-model experimentation is exactly what a heterogeneous, decentralized network of contributors is well suited to run in parallel.

4.4 Bittensor: a working market for machine intelligence

Bittensor is the substrate. Launched in 2021 with a fair launch, no pre-mine, and a 21-million-token hard cap, it is an incentive protocol in which miners contribute machine-intelligence work, validators score it, and the native token TAO is emitted in proportion to assessed value. The February 2025 "dynamic TAO" (dTAO) upgrade gave every subnet its own market-priced "alpha" token and liquidity pool, letting the market — not a central authority — allocate emissions toward the subnets it judges most valuable.

Selected indicators of Bittensor scale and maturity, early–mid 2026 [5–9].

Indicator	Value (early–mid 2026)
Active subnets	128, with a planned expansion toward 256
TAO market capitalization	~$2.5B–$3.4B (volatile)
Combined subnet "alpha" token market cap	~$1.1B–$1.5B (≈27% of TAO's)

Indicator	Value (early–mid 2026)
Emission schedule	21M hard cap; Dec 2025 halving cut daily emissions 7,200 → 3,600 TAO
Revenue signal	Leading subnets report seven-figure annual revenue from real usage
Institutional access	Grayscale Bittensor Trust listed; spot-ETF conversion filing pending

Two qualitative points matter more than any single number. First, the ecosystem is **maturing from emissions-farming toward real revenue**: several subnets now generate genuine income from paying users, and the market increasingly values subnets as revenue-generating assets rather than pure speculation. Second, Bittensor has already **proven the hard part** — Templar's decentralized 72B training run showed the network can coordinate serious, verifiable AI work, not merely inference.

4.5 The opening: no Bittensor subnet does this, and the field already moved

Bittensor's subnets span inference, distributed training execution, storage, data collection, financial prediction, and more. To the authors' knowledge, none operates autonomous training research as a competitive market — none asks miners to discover improvements to a training recipe and pays them for verified, merged contributions. Existing training subnets reward executing a training job; Ralph rewards improving the job itself. Within Bittensor, that position is unoccupied.

Outside Bittensor, it is not. This is the important honesty: automated training research is no longer rare or hypothetical. In 2026 Recursive beat the combined human-and-agent crowd on Karpathy's own autoresearch benchmark, and groups such as ScaleAutoResearch and PrimeIntellect run the same loop at Ralph's ~124M scale for a few thousand dollars per run. The field moved faster than any *—first-to-automate* claim could survive.

That is why Ralph's position is not —we run a research loop and others don't.— It is that running the loop is becoming cheap and common, and the durable asset is therefore not the search but the open, neutral, verifiable substrate the search runs on: an attested lineage of measured improvements bound to a private evaluation pool that a competitor copying the public recipe does not get (Section 5.4). A closed lab can out-search Ralph in any given week; it cannot be the shared, citable substrate that every team, including its competitors, builds on. Section 7 develops this against named competitors.

5. The Solution: Ralph

5.1 Design philosophy

Ralph takes autoresearch's proven loop and hardens it along the five axes Section 3.3 identified as missing, then wraps it in Bittensor's incentive layer. Three principles govern the design.

1. **Cheap search, expensive proof.** Most ideas are bad. The network must be able to reject them cheaply and spend real compute only on candidates that have already survived cheaper tests.
2. **Reward reliable improvement per compute dollar — not GPU budget.** The winner must be the miner whose contribution produces the most robust, scale-stable gain per unit of compute, so that a clever small operator can beat a brute-force large one.

3. **Assume miners will cheat, and design so honesty wins.** Where there is reward, there is gaming. Sandboxing, hidden evaluations, multi-seed confirmation, and patch auditing are not add-ons; they are the core of the protocol.

5.2 System architecture: private search, attested proof

The Ralph pipeline, two halves. Everything before submission is the miner's private business. The miner runs an autonomous research agent on their own infrastructure — any agent code, any LLM, any GPU, any training tricks during search — and iterates freely until they think they have a candidate worth proving. Ralph does not see, attest, or restrict the search loop; the protocol only sees what arrives in a submission.

The submission itself is a proof bundle: a patch, plus one or more *proof-test runs* of that patch inside the official Ralph Docker on a Confidential-Computing GPU. Each proof-test variant is a separate, more demanding run; a miner who wants their patch merged invests in the depth of their own proof, and is scored accordingly. The validator's role is to judge the resulting bundle cheaply.

Ralph proof-test variants and the validator's judgment steps. The miner's private search loop on their own hardware is invisible to the protocol; the only training the protocol attests is the proof-test run.

Proof variant	What the miner runs in the official Docker	What the validator does	Miner GPU cost	Validator cost
Proxy	Canonical short-duration training on a small-model size; produces checkpoint + log + attestation	Plausibility check on the training log	Minutes, 1 GPU	None — judgment only
Confirmation	Same patch, multiple seeds; produces N attested runs	Statistical agreement check across the miner's seeded runs	Hours	None
Scale	Canonical training at one or more larger model sizes from the track's model-size ladder	Plausibility check on the scaling curve	Hours–days, multi-GPU	None
Hidden eval	(Already submitted final checkpoint(s))	Inference-only evaluation against the rotating private set	None (already paid)	Single GPU, minutes
Audit (prob.)	(Already submitted)	Full re-execution of the proof test from the miner's declared seed and patch; score and trajectory must match within tolerance	Already paid	Hours–days (~10%, amortized)
Merge	—	Accepted patch is merged into the canonical recipe	—	—

A submission carrying only a proxy proof-test is permitted but downscored — the protocol cannot distinguish a real improvement from a small-model coincidence. A submission with confirmation and scale proof-tests on top earns full credibility. This is the same economic discipline frontier labs apply internally — small-model proxies first, multi-seed confirmation second, scale validation third — except here it is enforced by scoring, not by social norms, and the cost is borne by the entity with the incentive to bear it: the miner who wants their patch merged.

The hidden-eval step is the only hot-path GPU work the validator performs per submission. Because evaluation is inference, not training, it runs on a single GPU in minutes and is orders of magnitude cheaper than reproducing the run. The eval set is hidden, validator-owned, and rotated (Section 5.7), so a miner cannot have overfit to it.

The probabilistic audit is the integrity backstop. A small, randomly-selected fraction of submissions — plus all submissions within the noise-floor margin of the current canonical baseline, plus the first submissions from any new miner — are subject to a full proof-test re-execution. The validator re-runs the official Docker with the miner's declared patch, seed, and data manifest. Because GPU training is not bit-reproducible across runs even with fixed seeds and deterministic-mode flags — cuBLAS heuristic selection, non-deterministic kernels, and atomic-reduction ordering all introduce divergence — the audit does not require checkpoint hash equality. It requires **equivalence on what the network actually scores**: the validator's re-run checkpoint must achieve an eval-set score within the noise-floor margin of the miner's submitted checkpoint, and the loss-trajectory of the re-run's first 5–10% of steps must match the miner's submitted training log within a tolerance band. A re-run that materially underperforms either test indicates the submitted checkpoint did not come from the declared patch — grounds to treat the submission as fraudulent under Section 5.7.

Merge is unchanged: an accepted patch becomes part of the canonical recipe that every subsequent miner builds upon. The compounding property of the network is preserved, but the cost structure is appropriate to Bittensor: validator economics scale with the *number* of submissions judged, not with the *depth* of work each submission represents.

5.3 Miners: autonomous research organizations

A Ralph miner runs an autonomous research organization on their own infrastructure. The "organization" is the miner's private agent — one or more agents, configured by *program.md* files that describe roles, strategy, risk appetite, and which parts of the canonical-recipe repository the agent is allowed to touch. This is the abstraction autoresearch introduced — "a research organization is a set of Markdown files" — and Ralph inherits it without modification. The agent searches freely: any LLM, any sampling, any training code, any seeds, any data subsets, any hardware. Ralph places no constraints on the search; it pays for *what survives the proof test*, not for *how the candidate was found*.

Miners submit improvements as **patches, not whole codebases**. A submission is a base commit, a patch diff, an experiment configuration, and a declared expected improvement. Patch-based submission is what makes the system auditable: a validator applies the diff to a clean, known repository, so a miner cannot smuggle in logic that tampers with evaluation or scoring. Miners may compete across several contribution categories rather than one:

- **Model & optimizer:** architecture variants, normalization and activation changes, attention variants, optimizer and learning-rate-schedule improvements.
- **Data recipes:** better filtering, deduplication, curriculum schedules, mixture ratios, synthetic-data generation, contamination removal. Data quality often outweighs architecture tricks, so it is a first-class competition surface.
- **Post-training:** supervised-fine-tuning data selection and reinforcement / preference-tuning recipes.

- **Systems:** throughput, memory, and inference-latency optimizations — improvements that cost nothing in quality but raise effective compute efficiency.

The protocol meets the miner only at submission time. When the miner believes they have a winning patch, they prepare a submission bundle: the patch itself, an attested proof-test run of that patch through the official Ralph Docker (Section 5.4), and a signed PR against the canonical-recipe repository for the relevant track. Everything before that — the agent, the trial-and-error, the LLM costs, the failed candidates — is the miner's private investment, paid for in the miner's own time and compute. The miner is paid for discoveries, not for searching.

5.4 Validators: cheap judgment, hardware-backed trust

A Ralph validator does not own a training cluster. It owns a verification stack. For every submission the validator runs four cheap operations in sequence — any failure terminates the submission before the next operation is performed.

- **Operation 1 — Diff scan and bundle integrity.** Sub-second. The validator applies the submitted patch to the canonical baseline at the declared base-commit hash, then checks that no edit touched a restricted path — evaluation code, scoring code, data-split logic, logging, or the calibration-benchmark harness. The bundle's content-addressed hash must match the on-chain commitment. Submissions that fail this step are rejected with no further work.
- **Operation 2 — Attestation verification.** Seconds. The validator verifies the hardware attestation chain independently — not by trusting any centralized service — using locally-held Intel and NVIDIA root certificates, and checks the attested container measurement against the on-chain pinned value.
- **Operation 3 — Training-log plausibility.** Sub-second to seconds. The validator runs programmatic checks on the submitted training log: loss-curve monotonicity is reasonable; no NaN/Inf entries; throughput claims are internally consistent with the in-bundle calibration benchmark; gradient norms remain bounded; no suspicious checkpoint-resume points. This catches lazy fraud — replayed logs from a different run, hand-fabricated metrics that do not track a real training curve — without re-running anything.
- **Operation 4 — Hidden-eval inference.** Minutes on a single H100. The validator downloads the submitted checkpoint from its content-addressed storage location, hash-verifies it against the on-chain commitment, and runs the checkpoint through the validator's hidden, rotating evaluation set (Section 5.7). This is the only GPU work the validator performs in the hot path. Because inference is cheap and deterministic, two honest validators on identical eval sets produce scores that agree within the quantization grid.

The validator then reports its score into Bittensor's weight-setting consensus. The "decisively beats the king" margin from Section 5.7 prevents the canonical baseline from thrashing on statistical noise.

Why this is enough. The miner did the training. The validator establishes four things cheaply: (1) the patch is well-formed and respects restricted files; (2) the hardware that ran the training was a genuine Confidential-Computing GPU running the official Ralph container under the declared workload, with all attestations bound to the validator-issued nonce; (3) the training log is plausible and consistent with the calibration benchmark; (4) the resulting checkpoint achieves the claimed quality on a held-out evaluation the miner has never seen. None of these requires the validator to repeat the training.

Validator cost envelope. Concretely, the hot-path cost per submission is dominated by the hidden-eval inference: on the launch track (small LLM pretraining at nanochat-class scale) this is on the order of a few minutes on a single H100. At an expected submission throughput on the order of dozens per day per validator early in the network's life, a one-H100 validator is comfortably below saturation. The Stage 5 audit is the exception: a full re-train at the model-size ladder's upper end can take hours to days. Audits are therefore **amortized across the validator set rather than borne by each validator on every audited submission** — audit duty for any given submission is assigned by an on-chain randomness beacon to a single validator (or a small quorum), who is compensated through the validator emission stream for the GPU-time spent. The audit budget is a design parameter: at the baseline 10% rate it implies, in expectation, one full re-train per ten submissions across the whole validator set, not per validator. As tracks scale upward in model size, audit compute scales with them, and the audit-budget parameter is recalibrated through governance to keep the validator economics in band. The principle is that **validator-side compute is bounded by judgment work plus a calibrated audit share**, not by any miner's training depth.

The probabilistic audit closes the gap. Where the four cheap operations are necessary but not sufficient — for example, the miner could in principle submit a checkpoint that came from a *different* training run than the one declared by their patch — the Stage 5 audit catches it. A randomly-selected subset of submissions (biased toward king-margin and new-miner submissions) is fully re-executed. The miner is required to declare seed, data order, and library versions at submission time; the validator re-runs the patch with those settings and produces an independent checkpoint. Because bit-identical reproduction of a GPU training run is infeasible in practice, the audit checks **score-and-trajectory equivalence** rather than hash equality: the re-run's eval score must lie within the noise-floor margin of the miner's reported score, and the re-run's loss-trajectory over the first 5–10% of steps must track the miner's training log within a tolerance band. Failure on either test is treated as fraud, not error, and triggers the consequences in Section 5.7.

Hardware attestation: the single attested-execution tier

Ralph's scoring (Section 5.5) prices a contribution as quality improvement per normalized compute dollar produced under canonical training conditions. Two failure modes have to be foreclosed for that score to mean anything: the miner fabricating the result (Section 5.7's audit), and the miner running real training on cheap hardware while claiming expensive hardware — the denominator attack. Ralph addresses the denominator attack architecturally rather than statistically. There is one proof-test tier, and every submission to it carries a full hardware attestation chain. Submissions without a valid chain are rejected, not discounted.

The canonical proof-test environment. Every miner runs the official Ralph Docker image inside a Confidential VM on a Confidential-Computing-capable GPU — H100, H200, or B200 — hosted under Intel TDX or AMD SEV-SNP. The Docker's measurement is pinned on-chain. The CVM's signed attestation chain binds, in one nonce-coupled flow, the GPU report, the TDX (or SEV-SNP) quote, the container measurement, the data-manifest hash, and a rolling hash of the training log. Any deviation — non-official container, attestation missing, nonce mismatch, manifest mismatch — fails verification, and the submission is rejected with no recourse to a lower tier.

Ralph proof-test requirement.

Requirement	Treatment in scoring
Official Ralph Docker (on-chain pinned measurement) running inside a Confidential VM on a Confidential-Computing-capable GPU (H100 / H200 / B200) under Intel TDX or AMD SEV-SNP; full attestation chain valid per Section 5.4.	Compute claim accepted at the calibration-normalized H100 reference cost; no credibility factor.

Why one tier. The earlier two-tier design (verified at face value; unverified at a 0.5× credibility discount) priced the lie-about-hardware attack out of profitability statistically. The container-measurement layer adopted here forecloses it deterministically: a miner cannot produce a valid submission off the official image on a non-CC GPU, period. The 0.5× factor and its calibration are therefore retired. The trade-off is consumer-GPU exclusion at launch: a miner without access to a CC-enabled cloud cannot participate in the launch track. This is acknowledged honestly as Phase 4 work (Section 8). Once a credible cheaper trust primitive is available — fractional CC slots on commodity clouds, a hybrid attestation model, or a different per-track equilibrium — the network can lower the floor without giving up the architectural guarantee. Until then, the floor is the floor.

Miner search remains unconstrained. Only the proof step is subject to this requirement. A miner's private autonomous-research loop (Section 5.3) runs on whatever hardware they choose; the proof is what the network actually scores.

The proof-test handshake

A submission begins with a short on-chain handshake. The miner announces, on-chain, that they intend to prove a specific patch hash; the protocol returns a fresh nonce — derived from the next block's randomness beacon — that the miner's Docker must include as input to its TDX and nvtrust attestation reports. The nonce binds every subsequent attestation in the run to *this* proof-test attempt. The miner cannot run the proof first and then shop for a friendly validator to accept it: the nonce did not exist before the on-chain announcement, so any attestation chain that lacks it is rejected.

The miner then runs the proof test on their own CC-capable GPU. The Docker self-attests every Bittensor epoch (roughly 72 minutes, 360 blocks of 12 seconds), each attestation chaining the validator-issued nonce, the current container measurement, and a rolling hash of the training log accumulated since the previous attestation. At submission time the miner uploads the full chain. Validators verify it offline against locally-cached Intel and NVIDIA roots. *No validator needs to monitor the run live.* The chain itself is the verifiable record of what the GPU did during every epoch of the proof-test run; the cryptographic guarantee is the same as if a validator had been watching, without the operational cost of synchronous monitoring.

What the team operates

A subnet built on the attestation guarantee above requires that several artifacts be authoritative, signed, and version-coordinated across the network. The subnet-owner team operates these as core infrastructure:

- **The canonical baseline repository** for each track — the Git history that absorbs accepted patches and is the starting point every miner builds upon.
- **The eval pools** for each track — held-out tokens, benchmark mixes, FID image sets, and other track-native evaluation data, rotated under commit-reveal as in Section 5.7.

- **The miner Docker client.** Reproducibly built, signed, and published; its measurement is the network's anchor for trusted execution.
- **The validator client,** which validator operators run continuously to perform the four cheap operations of this section, sign weights, and audit when assigned.
- **The meaningful-failure verdict rule** applied by validators when scoring non-winning submissions for the 10% share (Section 5.6).

Upgrade cadence. Updates to the miner Docker client are coordinated network events. The team publishes a new image's measurement on-chain, along with a notice period before the old measurement is retired — a measurement is accepted by validators only after the notice window has elapsed, so the network does not split. Validator-client updates follow the same coordinated cadence. The team commits to a published release schedule (the working target is monthly minor releases plus security-driven out-of-cycle releases with longer notice periods), reproducible builds so any operator can verify the published measurement themselves, and a documented governance path for transferring this operational responsibility to a multi-party body as the network matures.

The attestation chain in detail

The verified execution layer is built on the only currently-defensible primitive for binding a workload to specific hardware: **NVIDIA Confidential Computing on H100/H200/B200, nested within an Intel TDX (or AMD SEV-SNP) CPU TEE quote, with a single validator-issued nonce flowing through both quotes**. The architecture is informed by the Manifold / Intel collaboration on Targon (Bittensor SN4), which is the first production deployment of this approach on Bittensor [19]; Ralph adopts the same primitives with two material adaptations.

The flow. A submission begins with the on-chain handshake described above: the miner announces a patch hash and receives a beacon-derived nonce. The miner's CVM, running the official Ralph Docker, constructs an Intel TDX attestation quote whose user-data field carries (a) the measurement of the running confidential VM, including the official Ralph image; (b) the nonce; (c) a content-addressed hash of the canonical recipe at the patch's base commit, plus the patch itself; (d) a content-addressed hash of the data manifest the canonical training will consume; and (e) the NVIDIA nvtrust GPU and NVSwitch attestation reports. The same nonce is included as input to the NVIDIA attestation, so the GPU's signed report carries it as well. Long runs re-attest every Bittensor epoch (roughly 72 minutes), with each attestation's user-data field updated to include the rolling hash of the training log accumulated since the previous attestation. At the end of the run the final attestation's user-data field also carries the hash of the full submission bundle: patch, checkpoint, training log, calibration result. The validator verifies the chain offline at submission time. The official Docker measures the canonical data manifest at startup and refuses to begin training unless the measurement matches the recipe's declared manifest, closing the audit-reproducibility gap.

Verification. Each Ralph validator verifies the attestation chain independently and offline, using locally-cached Intel and NVIDIA root certificates and Intel Provisioning Certificate Caching Service collateral. The validator checks: (1) the TDX quote's signature chains to the Intel root; (2) the NVIDIA nvtrust report's signature chains to the NVIDIA root [20]; (3) the same nonce appears in both quotes; (4) the user-data field of the final attestation matches the hash of the submitted bundle; (5) the measurement values in the TDX quote correspond to the expected CVM image. **Verification of any given submission requires**

no real-time call to a centralized service. This is a deliberate departure from Targon's architecture, which routes verification through a centralized key-broker service. For Ralph, full validator independence is preferred over the operational simplicity of a shared verifier. The one honest caveat is that Intel PCS collateral has a TTL and must be periodically refreshed from Intel's servers — a routine validator-side operation analogous to refreshing a TLS root bundle, performed out-of-band rather than per-submission. The per-submission verification path remains local.

Binding GPU, CPU, and workload, and the honest scope of what attestation proves. The Manifold and Intel architecture embeds the GPU attestation report inside the TDX quote's user-data field — structural nesting, not cryptographic binding. A sophisticated adversary could in principle splice a genuine TDX quote from one machine with a genuine NVIDIA report from another. Ralph closes this gap by requiring the same validator-issued nonce in both quotes; because the nonce is unique per submission and is supplied by the validator at run-start, neither quote can be drawn from a prior run on a different machine without breaking the nonce match. Adding the container-measurement layer eliminates a second attack surface: a miner cannot submit a checkpoint that was produced by anything other than the official miner client, because the TDX quote that accompanies the submission attests the exact container measurement, and the validator rejects any deviation. The much-discussed "borrow a public checkpoint" attack — substituting an off-the-shelf model as the output of a fabricated patch — is therefore foreclosed architecturally, not merely caught by audit.

The remaining honest caveat. NVIDIA Confidential Computing attests the GPU's state *at attestation time*, not continuously. The epoch-cadence re-attestation with rolling log hashes makes silent mid-run divergence costly to fake but does not make it cryptographically impossible. The attested-execution layer therefore gives a strong **probabilistic** binding over time — the longer the run, the more nonce-bound attestations the adversary must coherently forge — not an absolute one. The probabilistic audit (Section 5.7) is what closes this residual window for the most expensive submissions; for most submissions it is already overdetermined by the container-bound attestations alone.

Performance. Published measurements of NVIDIA Confidential Computing show under 5% wall-clock overhead on representative LLM workloads, with the impact concentrated on PCIe-bound communication rather than tensor-core throughput. For training workloads of the size Ralph's funnel typically considers, the overhead is below the noise floor of the scoring grid and does not affect competitive standing.

Failure semantics. A submission whose attestation chain fails verification is rejected outright. There is no fallback tier to absorb the failure. A miner uncertain whether their attestation environment will produce a clean chain should not submit; mining is permissionless and free, so a deferred submission costs the miner nothing. To avoid penalising honest miners for transient infrastructure issues, the validator surfaces the specific failure reason on rejection, and a miner whose chain fails for a documented infrastructure cause — Intel PCS collateral expiry, NVIDIA root certificate rotation, a coordinated network upgrade to a new container measurement — may re-submit within a grace window without burning a fresh on-chain handshake.

Validator capacity and the audit budget

A validator-light architecture is only credible if the validator workload it produces is actually bounded. Ralph's per-submission validator cost is dominated by two terms: a constant judgment cost (the four

cheap operations of this section), which runs in minutes on a single GPU, and a variable audit cost, which is a full re-train at the submission's declared scale. The first scales linearly with submission count and is comfortably absorbed by a single validator GPU at projected launch volumes. The second is the term that must be budgeted explicitly.

The audit budget. At a 10% baseline random-audit rate plus an estimated 5–10% mandatory audit rate from king-margin, probation, and community-flag triggers, a working assumption is that roughly **one in six submissions** enters the full audit queue. Audits are not run inline; they are queued and dispatched by validators with available capacity, with results returned within a published service-level window (initially 72 hours for small-track submissions, longer for larger scales). Validators that lack capacity for the audit queue do not reject submissions — they continue performing the cheap operations and defer to the network's audit pool, the same way Bittensor validators today specialise on different parts of their subnet's consensus duties.

Sizing. At small-LLM-pretraining scale (the launch track), an audit re-run is on the order of single-digit H100-hours. A validator running one H100 24/7 can process several audits per day, comfortably absorbing the audit load for a launch-phase miner population. At larger track scales the per-audit cost rises, but so does the available emission per round and therefore validator revenue; the network is calibrated so that validator emissions cover validator compute cost at the prevailing TAO price across all active tracks. If audit demand outruns validator capacity, audit-rate parameters — not validator economics — are adjusted by network governance. As with the other parameters in Section 5.7, these are working values to be refined during closed testnet.

The king lineage

Ralph maintains a single canonical recipe at any moment — the **king**. A submission is accepted only if it decisively beats the reigning king under the scoring rule (below), at which point it is crowned and becomes the parent every future submission must build on. The lineage is the chain of successive kings: a monotonically improving sequence of attested recipes, each one a measured step above the last.

The chain is not a Git ancestry check or an on-chain timestamp ordering. Each crowned king records a parent_king_attestation_hash — a flat pointer to the attestation of the king it dethroned. A challenger must reproduce its parent's evaluation on its own hardware and bundle a quorum-signed cache of that parent result alongside its own attested run; validators verify the chain by signature-checking the bundle rather than re-executing history. This keeps validator work minimal while binding every node in the lineage to the specific, hardware-attested result beneath it. Because the binding is an attestation hash and not a public commit, the chain cannot be reconstructed by an outside party who has only the public recipe.

The sealed-shard accumulator: why the lineage cannot be forked

The recipe at the head of the lineage is public; anyone can clone it. The evidence beneath it is not. Ralph evaluates every submission against a **private evaluation pool** — a set of held-out evaluation streams that never enter the miner's submission path. Each time the network crowns a new king, it seals a private hardness shard drawn from that pool and binds it to the king's attestation hash. The accumulated seal — the *sealed-shard accumulator* — grows with the lineage: depth-against-sealed-shards, not raw recipe quality, is the metric that a challenger must clear, and the pool a challenger is measured against is larger the deeper the lineage has gone.

This is the structural reason a fork cannot catch up. A competitor who clones the public recipe at the tip inherits the recipe but not the sealed pool: not the held-out streams, not the per-king hardness shards, not the attested record of which prior changes were tried against them and by how much each moved the metric. They can start a new lineage, but they begin at depth zero against a pool they must build from scratch, while Ralph's lineage continues to accumulate sealed evidence with every crowning. The moat is not the recipe and not secrecy of the method; it is the un-forkable, hardware-attested evidence the lineage is measured against.

A note on direction. The strict single-lineage invariant — every king must build on the prior king — is deliberately conservative, and it is monotonic by construction: it can only move along one chain. Recent automated-research results suggest multiple competitive basins exist at the ~124M scale, which a single chain cannot explore in parallel. The committed direction is a hybrid main-lineage-plus-parallel-branch extension (a lineage DAG) that preserves the sealed-shard guarantee while letting independent basins develop and later merge. This is forward work, not yet live; the launch network runs the single lineage described above. It is named here, and as an open question in Section 9, so the design's current limit is on the record.

5.5 Scoring: improvement per compute dollar

Ralph does not score raw model quality, because raw quality rewards whoever spent the most compute. It scores **the quality of a contribution**, normalized against its cost and its risk. Conceptually:

> **score = quality gain + benchmark gain + stability bonus**
> **− compute-cost penalty − complexity penalty − regression penalty**

Every term defends against a specific failure. The **compute-cost penalty** stops the network from rewarding brute force and keeps the winner "the miner who produces the most reliable improvement per compute dollar." The **stability bonus** and **regression penalty** stop an agent from chasing a tiny loss gain at the expense of throughput, memory, or training stability — a change that lowers loss 0.2% but cuts throughput 30% should score *negative*. The **complexity penalty** discourages unmaintainable hacks. Ralph can also run category leaderboards — best loss gain, best throughput gain, best memory reduction, best data recipe, best post-training recipe — because, as in real frontier labs, progress is the sum of many small specialized wins rather than one magic trick.

A hardware-independent compute unit. "Per compute dollar" only means something if the denominator is comparable across machines. Ralph normalizes compute to a reference accelerator — the NVIDIA H100-SXM 80GB — with the in-bundle calibration benchmark calibrated against published H100 reference timings; submissions are reported in normalized H100-hours rather than raw GPU-hours, so a discovery is rewarded for the efficiency of the idea, not the speed or expense of the hardware that happened to run it. Trustworthiness of the denominator is no longer a scoring parameter: it is a precondition. The proof-test attestation chain (Section 5.4) verifies that the run actually used the declared hardware, and submissions without a valid chain are rejected, not discounted. The earlier two-tier credibility factor (verified $\alpha = 1.0$, unverified $\alpha = 0.5$) is removed from the scoring expression.

5.6 The 90/10 split: rewarding winners and the search itself

A pure winner-take-all subnet pays only for the contribution that improved the baseline this round. That is simple, and Ralph preserves it as the dominant mechanism. But it also wastes the most abundant product of any honest research process — the carefully-run experiment that found a real boundary in the search space. In real science a rigorously-run negative result is not noise; it is the map of where not to look, and it is exactly the fuel the meta-layer (Section 5.9) and the experiment-record corpus (Section 6.2) depend on. Ralph splits per-round miner emissions to reflect both facts.

Per-round miner emissions.

Share	Goes to	Conditions
90%	Round winner	The single submission that produced a verified, multi-seed-confirmed improvement to the canonical baseline beyond the noise-floor margin (Section 5.4).
10%	Meaningful contributions	Submissions that did not win but cross the meaningfulness bar below, ranked by validator-assessed informativeness. If multiple qualify, they split the 10% by rank. If none qualify, the 10% is not paid out that round — it does not accumulate and does not redirect to the winner.

What "meaningful" means. Not every failed submission is a contribution. To qualify for a share of the 10%, a submission must clear a deliberate bar: it tests a **genuine, articulable hypothesis** (one coherent change with a stated rationale, not forty hyperparameters jiggled at once); it runs a **well-formed experiment** against the clean baseline with a complete training log and a passing calibration benchmark; it is **reproducible** (the same patch under the same declared conditions produces the same result, verifiable through the probabilistic audit of Section 5.7); it is **informative** — the result says *why* the change failed, with measured evidence (a specific regression on a named metric, a stability failure at a documented scale, a quantified interaction with another component); and it is **not a duplicate** of a failure the corpus already records — it must differ in substance, not be a near-copy farmed for the 10%.
Preventing duplicate-failure farming. Without a duplicate bar, the obvious attack is to flood the corpus with minor variations on the same known-bad idea and farm small rewards. A non-winning submission therefore earns a share of the 10% only when validators return a MEANINGFUL_FAILURE verdict: it must carry a structured rationale and a diff that materially differ from what the corpus already holds, and report a specific, measured failure (a named-metric regression, a documented stability failure at a stated scale) rather than a vague or null result. Submissions that duplicate an existing failure record, or that lack the required rationale-and-diff substance, earn nothing even if otherwise well-formed. The verdict rule and the corpus are public, so the meaningfulness call is reproducible rather than a per-validator embedding judgment.

Why this design pays for itself. It funds the experiment-record corpus that Section 6.2 packages as a product. It supplies the training data the meta-layer needs to evolve better research organizations. It softens the all-or-nothing dynamic that drives marginal miners off other subnets, broadening Ralph's active researcher base. And it is genuinely hard for a rival subnet to copy, because rewarding meaningful negative results is only coherent if the experiment-record infrastructure already exists to verify and de-duplicate them. What looks like paying for failure is in fact paying for **new information about the search space** — the actual product of any research network, whether positive or negative.

5.7 Anti-gaming and integrity

A network that pays for measured improvement attracts attempts to fake measured improvement. Ralph's defenses are layered: no single mechanism is load-bearing on its own.

Overfitting to the metric is defeated by the hidden, rotating evaluation set. On the LLM-pretraining launch track, the active eval at any time consists of approximately three million held-out tokens for validation perplexity, plus roughly fifteen hundred examples drawn from a held-out benchmark mix (subsets of MMLU, HellaSwag, ARC, and code/math tasks). Other tracks use eval pools native to their objective — held-out image sets and FID computation for a diffusion track, held-out speech samples and WER for an audio track — with sizes calibrated to that track's scoring noise. Across all tracks the same discipline applies: validators hold a pool ten times the size of the active subset off-network. The active subset rotates weekly, drawn from the pool by an on-chain randomness beacon under commit-reveal: validators commit hash(next_active_subset) on-chain at the end of each week and reveal at rotation. The pool itself refreshes monthly, with the oldest tenth replaced by freshly-collected held-out data. Every merged patch is re-scored against the *next* rotation's eval as a canary; a merged patch that regresses materially under fresh evaluation is demoted and the previous canonical baseline restored.

Lucky single-run results are defeated by the multi-seed discipline requirement at Stage 2. A submission accepted on a single seeded run is downscored relative to a submission that demonstrates repeatability across multiple seeds. The "decisively beats the king" margin — set empirically by running the unchanged baseline approximately one hundred times across the active eval to measure score variance — is the noise floor a submission must exceed before being eligible for merge.

Tricks that do not scale are defeated by the model-size discipline at Stage 3. A miner who wishes their patch to be accepted at the larger end of the model-size ladder must submit results at multiple scales; a patch that improves at small scale but regresses or fails to extrapolate is rejected at merge time.

Tampering with evaluation or scoring is defeated structurally rather than by inspection. The hidden eval set is held off-network by validators and never enters the miner's submission path at all — the validator scores the miner's submitted checkpoint against eval data the miner has never seen. The canonical training code, calibration harness, and data-split logic live inside the official Ralph Docker and are part of the on-chain pinned container measurement, so any modification produces a different measurement and the attestation chain is rejected. The restricted-file diff scanner additionally rejects any patch that touches them before the proof run begins. The patch in a submission can only modify what the recipe permits it to modify (model code, optimizer code, data-mixture configuration, etc.).

Plagiarism of another miner's submission is defeated by the structured experiment record (Section 5.8) combined with on-chain submission timestamps verified through the hosting platform's API. A later near-duplicate must beat the original by a clear margin to displace it.

False compute claims and hardware misrepresentation are foreclosed by the proof-test attestation chain (Section 5.4). Every submission binds its run to a specific genuine Confidential-Computing GPU through the nonce-coupled GPU and CPU quotes; a miner cannot claim H100-class compute while running on lesser or non-CC hardware, because the GPU's own signed report would contradict the claim and the submission is rejected before scoring. There is no statistical discount to evade because there is no lower tier to fall back to — the attack does not produce a scoreable submission at all.

Lying about a training run that did not actually happen is defeated by the probabilistic audit. Audits run on:

- 10% of all submissions, selected by an on-chain randomness beacon at acceptance time;
- 100% of submissions within the noise-floor margin of the current canonical baseline — high-stakes decisions cannot afford fraud;
- 100% of submissions from a hotkey that has not yet completed three honest audited submissions (probation);
- 100% of submissions flagged by another miner or validator, with a bounty paid on confirmed fraud.

A submission selected for audit is fully re-executed by the validator using the miner's declared patch, seed, data order, and library versions. The re-run's eval score must agree with the miner's reported score within the noise-floor margin, and the early loss-trajectory must match the miner's training log within tolerance. Failure on either test is treated as fraud, not error. Ralph uses the consequences Bittensor gives natively, layered for severity:

- **Forfeiture of the round's emission.** A miner earns alpha only when validators score them as the winner or a meaningful contributor. A submission caught in fraud earns nothing that round, retroactively. The miner has already paid the compute cost of fabricating the submission; that cost is sunk regardless.
- **Zeroed weights and blacklist.** Validators set the offending hotkey's weight to zero and refuse to score future submissions from it. On Bittensor this is the strongest native consequence available to validators, and it ends the hotkey's ability to earn.
- **Reputational cost.** The fraud event is recorded in the public experiment-record corpus under the hotkey's permanent attribution. A miner team running honest infrastructure cannot detach from this record.

Why this is sufficient. Ralph does not require miners to post collateral, hold alpha, or stake capital to participate. Mining here is permissionless and free: a miner earns only when they win. The fraud math therefore rests on three quantities the attacker cannot avoid: their own sunk compute cost on the fraudulent submission (S), the recycle fee paid to register the hotkey (F), and the future emission they lose if caught and blacklisted (E). At a 10% baseline audit rate, with the audit catching the dominant fraud attacks (see below), the expected return of a fraud attempt is:

$$EV(\text{fraud}) = 0.9 \cdot R_{\text{winner}} - 0.1 \cdot (S + F + E)$$

Deterrence holds when $R_{\text{winner}} \le (S + F + E) / 9$. This condition is comfortably satisfied for any established miner: future-emission stake E alone is large relative to a single round's winner emission for any miner planning to participate for more than a few rounds, and for miners running real training S is non-trivial on top.

The genesis-phase edge case. The narrowest fraud-economics question is a brand-new hotkey with $E \approx 0$ and a recycle fee F that scales down with low launch-phase activity. That edge case is closed not by the audit but by the attested execution layer of Section 5.4: a submission's checkpoint is bound by hardware attestation to a run that started from the canonical baseline inside the official miner container, so the dominant cheap-fraud attack — substituting an off-the-shelf public checkpoint as the supposed output

of a fabricated patch — cannot produce a valid attestation in the first place and is rejected before scoring. To submit anything that the network can score, a miner must actually run the official client end-to-end, which forces *S* to be non-trivial. The cheap-fraud path is closed at the protocol boundary, not at the audit.

Reputation discount for honest miners. A miner with a long record of honest, audited submissions earns a reduced random audit rate of 5%, a small efficiency dividend that resets on any flag. The mandatory 100% audit triggers (king-margin, probation, community flag) are unaffected by miner record.

Note on calibration. *All numerical parameters in this section — eval set sizes, rotation cadence, audit rate, probation length, noise-floor margin, and the 90/10 emission split — are working values to be empirically refined during closed testnet. The principles are intended to be stable; the exact numbers will move as the network produces its first hundred submissions and we observe the actual fraud-detection rate, score variance, and submission-to-merge funnel ratio.*

5.8 Auditability: the experiment record

Every submission carries a structured record — idea, files changed, expected benefit, compute budget, baseline and new scores, seeds, cost, and acceptance decision. Without this discipline a research network degrades into an unauditable pile of code edits. With it, every emission the network pays is traceable to a specific, reproducible, attributed contribution. An illustrative record appears in Appendix A.

5.9 The meta-layer: evolving the research organization

The deepest idea in Ralph is also the one Karpathy and his podcast host singled out as the natural next step. If a research organization is just a set of *program.md* files, and those files are just code, then **the organization itself can be optimized**. One contest format raised in the autoresearch discussion is precisely this: have contributors submit different *program.md* configurations, give them identical hardware, and measure which research organization produces the most improvement — then feed that data back to a model and ask it to write a better *program.md*. This meta-optimization is not hypothetical: evolutionary refinement of an agent's own prompts and code is already the core mechanism of widely adopted production agents such as Nous Research's Hermes Agent [18]. Ralph's contribution is to run it as an open, verified competition rather than inside a single closed system.

Ralph is designed to host exactly this meta-competition as a later phase. The first phase competes on patches to the training recipe. A subsequent phase competes on **research-organization design** itself — miners are scored on the improvement-per-compute their *program.md* achieves, and the network accumulates data on which organizational strategies work. This is recursive self-improvement made into an open, measurable, ownable market: the subnet improves the models, and then improves the process that improves the models.

5.10 Tracks: how Ralph scales across model classes and modalities

Ralph does not assume one universal training recipe. Different model classes and modalities have different objectives — validation perplexity for an LLM, FID for an image generator, word-error-rate for a

speech model — different eval regimes, and different definitions of "a better recipe." Treating them as one would be neither useful nor honest. Ralph therefore organizes competition into **tracks**, and the shared network of validators, miners, and infrastructure runs them in parallel.

Anatomy of a track.

Component	What it is
Canonical baseline	A Git repository defining the starting model class, training code, data pipeline, and reference hyperparameters for this track. Every accepted patch in the track is merged here.
Eval pool	A validator-held, hidden, rotating set of evaluations native to the track's objective — perplexity tokens and benchmark mixes for LLM tracks; held-out image sets and FID computations for diffusion tracks; held-out speech samples for audio tracks.
Scoring weights	Track-specific weights on the quality / cost / stability / complexity terms of Section 5.5. A coding-post-training track might weight benchmark gain higher than throughput gain; a small-LLM-pretraining track might do the opposite.
Model-size ladder	The set of scales at which Stage 3 of the funnel verifies extrapolation. Track-specific because the relevant scaling regime is different for a 1.5B pretraining track and a 70B post-training track.
Miner population	The miners actively competing in this track. A miner may compete in several tracks; emissions for the track flow only to miners scored in it.

What is shared across tracks. The validation funnel, the validator software, the hardware-attestation chain, the experiment-record schema, the dTAO subnet, the alpha token, the buyback program, and the meta-layer are all *one* set of tools used by every track. What is track-specific is the baseline repo, the eval pool, the scoring weights, the model-size ladder, and the miner population. This is what makes Ralph one network rather than ten.

The experiment-record corpus is cross-track. Every submission in every track lives in the same searchable record, with track membership as one of many indexable fields. This is the artifact that makes Ralph useful even when no single track exactly matches a researcher's problem — they can query the corpus across tracks for patches validated under similar conditions and inherit techniques the network has already de-risked elsewhere.

Launch and expansion. Ralph launches with a single track: **small LLM pretraining** — the autoresearch lineage. It is the most concrete, the most cheaply verified, and the one whose miner population already exists. New tracks are added by network governance once three preconditions are met: an open, well-defined evaluation regime exists; a credible held-out eval pool can be assembled and rotated; and a community of miners has signalled willingness to compete. Plausible next tracks, in approximate order of accessibility: **LLM coding post-training**, **LLM math post-training**, **small diffusion / image generation**, **small audio**. Larger and more capital-intensive classes (video generation, large-scale post-training) follow as the network's compute footprint grows. Each new track is a category, not a fork: it shares Ralph's validators, token, and corpus, and adds to the experiment record that benefits every existing track.

6. Economic Model

Note: Token mechanics below are a design sketch for discussion, not a final specification, and are subject to Bittensor protocol rules and to legal review. No token sale is described or offered by this document.

6.1 Framing

Ralph is a Bittensor subnet operating under dynamic TAO, live on mainnet at netuid 40, with its own alpha token. As established in Sections 5.6 and 5.7, mining requires no collateral, bond, or token-holding to participate; miners earn alpha only when validators score them as a winner or a meaningful contributor. The economic strategy of this section is revenue-driven rather than emission-driven: Ralph is designed to be funded by what the network sells, not by the rate at which it issues new tokens. The mechanics that follow — revenue streams, programmatic buybacks, milestone-paced communication, and concrete proof artifacts — are the load-bearing parts of that strategy.

6.2 Revenue: the ralph-diffs data license

Ralph is designed to be a business, not a subsidy, and it has one primary product. As established in Section 1.1, the durable artifact the network produces is ralph-diffs: the structured, attested corpus of every recipe change the network has evaluated — the diff, its measured effect across the evaluation ladder, its multi-seed variance, the hardware-attestation hash, and a pointer to the parent it built on. This is the artifact frontier labs generate internally and never release, and it is exactly the training signal a recipe-proposing model learns from. It is the thing applied-research teams will pay for.

The market is real and growing: machine-learning-as-a-service was valued at roughly $46–62B across 2025–2026 and is widely projected to compound above 30% annually, with model training and tuning its single largest segment at about 30% of spend [17]. The recurring pain it pays to solve — finding the recipe, configuration, or data mix that trains a model better and cheaper without burning a fortune in trial-and-error compute — is precisely what the lineage produces, run after run. Revenue streams, in order of how directly they monetize that artifact:

1. **The ralph-diffs data license — the primary product.** The diff corpus ships as a regularly-updated dataset (recipe diff, evaluation-ladder delta, multi-seed variance, attestation hash, parent reference). It is published under a share-respecting open data license for public use and citation, and offered to applied-research teams and labs as a commercial ingestion license for training directly on it. The recipe at the head of the lineage stays a free public good; the accumulated attested evidence beneath it is the licensed, monetizable artifact.
2. **Recipe and corpus access subscriptions.** Hosted, queryable access to the canonical baseline and the corpus for teams that want a leading recipe and the evidence behind it without ingesting the full dataset or running the search themselves.
3. **Enterprise and private deployments.** Licensed deployment of the Ralph loop inside an organization's own infrastructure, run as a managed service for teams that cannot place workloads on a public network.
4. **Sponsored optimization and model products — later.** Once the lineage and the dataset business are established, a sponsor can fund the network to drive a specific training metric, and the

maturing model lineage (Ralph-1, Ralph-2) becomes a product line in its own right. These are follow-on streams, not the launch wedge.

The plan is honest about timing. Revenue is not present on day one. The first paid artifact is the —ralph-diffs— license, which requires a lineage with enough accumulated, attested diffs to be worth ingesting; the network targets a first data-license pilot in the months after the lineage begins compounding on mainnet. Exact timing depends on lineage depth and on closing a first paying team; this whitepaper does not assert one is committed at signing.

6.3 The buyback program

Revenue is recycled into the token. Income received by the subnet — in TAO, or in stablecoin and converted — is used to programmatically purchase Ralph alpha on the subnet's own dTAO pool. This creates buy pressure tied to real product usage rather than speculation, a revenue-funded analogue of the auto-staking buyback Rayon Labs operates on Chutes (SN64) [21]. The two are not identical: Chutes' mechanism auto-stakes a portion of emissions, whereas Ralph's buyback is funded by external revenue from the ralph-diffs license. Both create alpha demand not driven by speculation; Ralph's is contingent on real revenue, which is the honest distinction. The result is a closed loop: verified research output earns revenue; revenue buys alpha; a healthier alpha token attracts staking and a larger emission share; a larger emission share funds more research capacity.

Every buyback is an on-chain transaction and will be reported on a regular published schedule. The objective is a subnet that funds its own existence — token sustainability and alignment, not price management.

An honest caveat. The broader Bittensor ecosystem is in a live debate about whether subnet revenue is yet large enough to support the emissions issued against it; published critiques have argued that even the largest revenue-generating subnets remain heavily emission-subsidized at current prices. Ralph does not assume otherwise. The buyback program is presented as the mechanism through which revenue eventually offsets emission, not as a claim that this is solved at launch. The case for Ralph rests on the underlying assets (the recipe and corpus) being genuinely valuable, the funnel being genuinely compute-efficient, and revenue ramping faster than emission inflation — each of which is testable against the network's own published cadence rather than asserted in this document.

6.4 Network growth and the milestone cadence

A subnet's emission share follows market attention and staking conviction, which in turn follow visible, credible evidence that the subnet works. Here Ralph has a structural advantage most subnets lack: its normal operation produces a continuous stream of verifiable, dated results. Every accepted patch is a real, independently checkable improvement to the canonical baseline. Every benchmark threshold crossed, every improvement that survives the next model-size tier, every new competition category, and every meta-layer milestone is a genuine event rather than a manufactured one.

The plan is therefore a disciplined public cadence: regular, scheduled reporting of accepted improvements and their measured gains, canonical-baseline benchmark progress, buyback transactions and treasury status, and roadmap milestones delivered. Because the underlying results are verifiable, this is communication rather than promotion — the subnet earns attention by publishing proof. A research network that compounds in quality is, by construction, a network with something new and true

to report on a regular schedule; Ralph is built so that its growth narrative and its actual output are the same thing.

6.5 Demonstrations: the proof artifacts

Verifiable milestones are most persuasive when they take a concrete, inspectable form. A market of token-holders — many of them non-specialists — is convinced less by a description of the mechanism than by an artifact they can see, download, or re-run for themselves. Ralph therefore commits to a deliberate slate of public demonstrations, each chosen to be verifiable by anyone, legible at a glance, and hard to fabricate.

Planned demonstrations, in approximate sequence.

Artifact	What it shows
Ralph-1 reference model	A real model trained by the evolved canonical recipe, released with weights, recipe, and a one-command reproduction. The headline claim is efficiency — matching a public baseline at materially lower compute. Periodic releases (Ralph-1, -2, …) make the compounding gain visible over time.
Ralph Live dashboard	A public, always-on view of the canonical baseline's benchmark trajectory, every accepted patch and its measured gain, cumulative compute saved, category leaderboards, and on-chain buyback transactions — the continuous, real-time proof signal.
Cross-domain demonstration	The same loop applied beyond language models — for example a vision, diffusion, reinforcement-learning, or scientific-ML model — proving the method generalizes, exactly as Shopify's non-training use of autoresearch did.
Sponsored-bounty result	A named external sponsor's training metric improved through the bounty mechanism, with a verifiable before/after — evidence that the network's output has real, paying demand.
Open research output	Periodic technical reports and the audited experiment-record corpus released openly — citable contributions to the field, including negative results, rather than product updates alone.

The headline test: transfer credibility (B6). The most important demonstration is not a model or a dashboard — it is a falsifiable bet the network has pre-registered against itself. Ralph's entire premise is that selection at small ~124M scale produces signal that means something. That is an empirical claim, and it can fail. So before scaling it, Ralph committed in advance to a public test: does a recipe's score on Ralph's evaluation ladder actually predict its standing against an independent external reference model?

The pre-registered criterion is a rank correlation of Spearman ≥ 0.6 between Ralph ladder scores and performance against a fixed external reference (OLMo-2-1B at 30B tokens), with the GO decision requiring both the point estimate to clear 0.6 and the lower bound of its 95% confidence interval to stay above 0.5. The threshold, the reference model, and the analysis were fixed before the result was known, and the raw outcome is published either way — a GO scales the network as designed; a NO-GO triggers the fallback path in Section 8 rather than a quiet redefinition. A research instrument earns trust by naming the test it could fail and then running it in the open; this is that test.

Each artifact is independently verifiable. The subnet earns attention the same way it earns revenue: by shipping checkable proof of the value it creates.

7. Competitive Landscape

Ralph should be judged honestly against what already exists, and a great deal now exists. Two things are true at once: the submit-verify-merge pattern is proven on Bittensor, and autonomous training research itself is no longer rare. The honest question is therefore not whether the mechanism works, nor whether anyone else does automated research — they do — but what is durable when the search is cheap and contested. The answer is the substrate, not the search.

Where Ralph sits relative to adjacent efforts.

Category	Examples	How Ralph differs
Bittensor improvement-market subnets	TrajectoryRL (SN11); Distil (SN97); Ninja (SN66); QUASAR (SN24)	These prove the submit–verify–merge pattern works on-chain, but each optimizes a fixed artifact — a prompt pack, a distilled model, a CUDA kernel. Ralph applies the pattern to the training process itself, where verification is expensive and noisy and demands a multi-stage funnel none of them needs.
Autonomous research tools	autoresearch; pi-autoresearch; AI Scientist; AutoSOTA	Single-user, trusted tools with no incentive layer. Ralph adds a decentralized market, staged adversarial verification, and a shared compounding baseline.
Decentralized training networks	Prime Intellect; Nous Psyche; Pluralis	They reward executing a training job. Ralph rewards discovering improvements to the job, then merges them into a recipe every future job inherits.
Automated-research labs	Recursive; ScaleAutoResearch; PrimeIntellect; frontier-lab internal RSI programs	These already run the autoresearch loop — Recursive beat the human-and-agent crowd on Karpathy's own benchmark; others run at Ralph's ~124M scale for ~$2k/run. The search is not the moat. Ralph is the open, neutral substrate beneath them: the attested provenance and un-forkable sealed pool none of them publishes.

What the Bittensor improvement markets already prove. This is a strength, not a threat. Subnets that accept miner improvements as pull requests bound to on-chain commitments, and merge the winner into a base repository every later miner builds on, demonstrate the compounding-baseline mechanism in production. Others run competitive model and kernel improvement scored on objective, reproducible metrics. Ralph's core loop is therefore de-risked: submit-verify-merge is known to function on-chain.

What the autoresearch labs already prove. Recursive, ScaleAutoResearch, and PrimeIntellect have shown that an automated research loop can match or beat human-and-agent crowds on exactly the small-scale training benchmarks Ralph competes on, and can do it cheaply. Ralph does not dispute this; it depends on it. A world where many actors can run the loop is precisely the world in which the scarce asset stops being the recipe and becomes the *attested record of what the loop found*.

Ralph as the ecosystem's research layer. The sharpest framing is not Ralph versus these efforts but Ralph beneath them. Training-execution networks, model subnets, and applied-research teams all consume a training recipe; Ralph produces one, and an improving one, together with the diff corpus behind it. Its canonical baseline and —ralph-diffs— are inputs other subnets and outside labs can adopt and cite directly. That positions Ralph as upstream infrastructure for the training ecosystem rather than one more competitor for a slot.

Defensibility: the un-forkable provenance. Ralph crowns a single canonical recipe — a king — each round, and the public recipe at the head of the lineage can be cloned by anyone in an afternoon. The moat is therefore not the recipe, and not secrecy. It is the sealed-shard accumulator beneath the lineage (Section 5.4): each time the network crowns a king, it seals a private hardness shard bound to that king's attestation hash. A competitor can fork the public recipe, but it inherits none of the private sealed pool the lineage was measured against, and cannot reconstruct the attested record of which changes helped, hurt, and by how much. The compounding artifact a late entrant cannot retroactively acquire is the provenance itself: an unbroken, hardware-attested chain of measured improvements, bound to evidence that does not travel with a fork.

8. Roadmap

Ralph is built one layer at a time — each phase is usable on its own and de-risks the next. Phases 0 and 1 are complete: the network is live on Bittensor mainnet, netuid 40. The pivot of the roadmap is the B6 gate in Phase 2, which decides honestly between scaling the substrate and re-scoping the central claim. Timelines beyond the gate are indicative and conditional on it.

Phase	Status	Focus	Milestones
0 Foundations	**DONE**	Harden the loop	Multi-stage funnel and sandbox built on the autoresearch baseline; structured experiment record; reproducible scoring; king lineage with attestation-hash chaining; closed testnet.
1 Mainnet launch	**LIVE · netuid 40**	Go live on Bittensor	Subnet live on Bittensor mainnet (netuid 40); canonical baseline + public/private evals (CORE-22 + private-hard over a sealed evaluation pool); miner & validator clients; PR/diff submission; single ~124M lineage; single attested-execution tier; Ralph-1 reference model and the Ralph Live dashboard.
2 The B6 gate	**NEXT**	Prove transfer credibility	Run the pre-registered transfer-credibility test (Spearman ≥ 0.6 vs OLMo-2-1B at 30B tokens; GO requires point estimate > 0.6 and 95% CI lower bound > 0.5). Publish the raw result either way. This gate decides the path below.
3a Scale (on GO)	**CONDITIONAL**	Full v0.11: scale the substrate	On a GO: ship the sealed-shard accumulator at full strength; begin the ralph-diffs data-license pilot; CSDP Pareto scoring (cross_scale_v1) as the king rule; model-size ladder; revenue-funded alpha buybacks; first sponsored optimization result; lineage DAG / parallel-branch design to address multiple competitive basins.
3b Fallback (on NO-GO)	**CONDITIONAL**	Re-scope, don't redefine	On a NO-GO: do not scale the small-scale-selection claim. Re-scope to the eval scales where the rank correlation does hold, harden the evaluation surface, and re-run the gate before committing to the full data-license business. The fallback is named in advance so the bet stays honest.

4 Decentralization	**FUTURE**	Research-as-a-service; hand off control	Recipe and ralph-diffs products at scale; enterprise and private deployments; periodic —State of the Baseline— reports; buyback at scale; a hybrid attestation model bringing consumer-grade GPUs (4090 / 5090-class) into the attested tier (one forward line, not a launch dependency); progressive transfer of baseline maintenance, miner-image release, and eval-pool rotation to a multi-party governance body.

The shape of the plan is deliberate. Everything through Phase 1 is shipped and verifiable on-chain. Phase 2 is a single falsifiable test the project has bound itself to in public. Phases 3a and 3b are the two honest branches out of that test — scale if the evidence supports it, re-scope if it does not. The mechanisms that are not yet live (the full sealed-shard accumulator, CSDP as the active king rule, the lineage DAG, consumer-GPU attestation) are named as committed forward work, not described as already running.

9. Risks and Mitigations

A credible whitepaper states its risks plainly. Ralph's most material risks, and the design choices that address them:

Principal risks and mitigations.

Risk	Why it matters	Mitigation
Metric gaming	Miners optimize the measurement, not the model.	Hidden + rotating evals; contamination checks; patch-only submission; restricted-file scanner; no-label sandbox.
Experiment noise	ML results are stochastic; a lucky run looks like a discovery.	Mandatory multi-seed, multi-data-order confirmation before acceptance.
Validation compute cost	Verifying every submission could be prohibitively expensive for validators.	Validator-light architecture (Section 5.4): miner-borne training, single-GPU inference for hot-path scoring, a calibrated audit budget amortized across the validator set rather than per-validator.
Scale mismatch	Small-model gains may not transfer to large models.	The pre-registered B6 transfer test (Spearman ≥ 0.6 vs OLMo-2-1B at 30B tokens; Sections 6.5, 8) gates this directly: if small-scale ladder scores do not predict external standing, the network re-scopes rather than scaling the claim.
Transfer credibility unproven	Ralph's premise — that small-scale selection produces signal that means something — is an empirical claim that can fail.	Named and pre-registered, not assumed: the B6 gate (Spearman ≥ 0.6 vs OLMo-2-1B) is run in public with the result published either way, and a NO-GO triggers the Section 8 fallback rather than a quiet redefinition.
Autoresearch commoditization	Running the research loop is now cheap and contested (Recursive, ScaleAutoResearch, PrimeIntellect), so being first to a better recipe is not durable.	The thesis already assumes this: the moat is the un-forkable attested provenance and the sealed evaluation pool (Section 5.4), not the search. Ralph is positioned beneath the

Risk	Why it matters	Mitigation
		autoresearch labs as the substrate, not against them as a faster searcher.
Single-team dependence	A subnet reliant on one operator is fragile — the Covenant AI / Templar exit of April 2026 cut TAO ~20–25% and showed the damage when a key team dumps owner alpha and leaves.	Open-source everything; multi-validator design; published baseline so the project survives any single operator; progressive decentralization of governance.
Token & regulatory	Subnet alpha tokens are volatile; their legal classification is unsettled.	No reliance on token price for core function; legal review; revenue-funded buyback as the durable demand offset; clear disclaimers.
Diminishing returns	A well-tuned baseline yields fewer easy wins over time.	Expand competition surface — data, post-training, systems, then the meta-layer — so new frontiers keep opening.
Single-lineage monotonicity	The strict —every king builds on the prior king— invariant explores only one chain. Recent results suggest multiple competitive basins exist at ~124M scale, which a single lineage cannot develop in parallel; the network could lock onto a local basin.	Committed lineage-DAG / parallel-branch extension (Section 5.4) that preserves the sealed-shard guarantee while letting independent basins develop and later merge. Stated as forward work; the launch network runs the single lineage.
Official Docker as trust anchor	With the single-tier design, the official Ralph Docker is the trust anchor for the entire network: a flaw that lets user-supplied weights into the signing path would invalidate every submission produced during the vulnerable window.	Reproducible builds and published measurements so any operator can verify the image; third-party security audits before each release; coordinated multi-week notice for image rotations; documented security-out-of-cycle release path with abbreviated notice when the flaw is in the old image (Section 5.4 upgrade cadence); medium-term governance path for transferring the signing key from the team to a multi-party body.
Consumer-GPU exclusion at launch	The single-tier requirement excludes miners without access to CC-enabled cloud GPUs from the launch track. This narrows the launch miner population and depends on CC cloud pricing for participation cost.	CC cloud capacity is general-availability on Azure NCC, GCP Confidential VMs, and selected neoclouds, and prices have tracked standard H100 rentals within ~10–20%. Phase 4 (Section 8) commits to a hybrid trust path that lowers this floor as a credible cheaper primitive becomes available.
TEE side-channel and root-key compromise	If Intel TDX, AMD SEV-SNP, or NVIDIA Confidential Computing is broken by a vendor-level vulnerability or root-key compromise, the attestation chain no longer proves what it claims to prove.	Defense in depth via the probabilistic audit; coordinated emergency container-measurement rotation on disclosure; participation in Intel and NVIDIA security-advisory channels; out-of-scope threats are documented in Section 5.4 honestly rather than denied.

10. Conclusion

Two facts about AI in 2026 sit uncomfortably together. The first: the most valuable capability in the field — a system that does research — is no longer hypothetical or rare. In the span of a year it went from Karpathy's overnight proof-of-concept to Recursive beating the combined human-and-agent crowd on that same benchmark, with groups like ScaleAutoResearch and PrimeIntellect now running the loop at the same small scale for a few thousand dollars a run. The second: the *evidence* those systems produce — which recipe edits worked, under what conditions, and by how much — is still being generated privately and thrown away, or kept behind the same closed doors as everything else.

Automated research got cheap. That is exactly why the durable thing is not the search.

When anyone can run the loop, the asset is no longer being first to a better recipe — it is the **provenance**: an unbroken, hardware-attested chain of what was tried and what it did, bound to an evaluation pool that grows with the work and that a competitor copying the recipe does not get. A closed lab can out-search Ralph on any given week. It structurally cannot be the open, neutral, verifiable substrate that every team — including its competitors — can build on and cite. That is the position no incumbent can take, and it is the one Ralph is built to hold.

So the right way to understand Ralph is by what the substrate leaves behind. As the lineage grows, it accumulates three things that did not exist before:

- **An open recipe** that has absorbed every accepted improvement and is materially better than the day-zero baseline — one repository any lab, startup, or researcher can clone.
- **ralph-diffs** — on its way to the largest open, structured corpus of *attested* training experiments anyone has released: hypotheses, diffs, conditions, results, negative results, and the cryptographic proof each one actually ran. The kind of dataset a recipe-proposing model is trained on.
- **A model lineage** — Ralph-1, -2, … — open weights that make the compounding improvement legible and verifiable to anyone.

The subnet and its token are what fund those artifacts; they are not what Ralph is.

Ralph is willing to be measured on this. Before scaling the claim that small-scale selection produces real signal, the network pre-registered a public test: whether a recipe's score on Ralph's ladder actually predicts its standing against an independent reference model, with the threshold and the analysis fixed in advance and the raw result published either way. A research instrument earns trust by naming the test it could fail — and then running it in the open.

The technology is ready and contested. The substrate is not. This document is the invitation to build it.

> **The thesis, restated.** Recursive self-improvement — the closely-guarded crown jewel of every frontier lab — is being automated in public, and getting cheaper every month. The scarce thing is no longer the search; it is the *attested record* of what the search found. Run on a verified market, under hardware-attested rules, bound to an eval pool that cannot be forked, that record becomes a public good no closed lab can produce: a recipe the world can clone, a corpus the world can train on, and models the world can run. That is how the most valuable loop in AI stops being a private advantage and becomes shared infrastructure.

Glossary

This document straddles two technical communities. The glossary disambiguates terms a reader from either community may want defined.

Bittensor and dTAO

Bittensor. An incentive protocol launched in 2021 that pays a global, permissionless network of *miners* to produce machine-intelligence work and *validators* to score that work. Native token TAO; hard cap 21 million.

Subnet. A specialized market within Bittensor with its own task, its own miners, its own validators, and (since dTAO) its own token. There are 128 active subnets at the time of writing, with a planned expansion to 256.

dTAO (dynamic TAO). A February 2025 protocol upgrade that gave every subnet a market-priced "alpha" token paired with TAO in an automated-market-maker pool. Staking TAO into a subnet mints alpha; the alpha price in TAO is the ratio of the pool reserves. The market — not a central authority — decides what share of network emissions each subnet receives.

Alpha token. A subnet's native token under dTAO. Its price in TAO is set by the AMM. Miner emissions are paid in alpha.

Hotkey, coldkey. A Bittensor account has two keypairs: a hot key used for active operations (mining, validating, weight-setting) and a cold key holding the actual stake. Slashing or blacklisting affects the hotkey.

Recycle fee. The TAO fee paid to register a new hotkey on a subnet, scaling with subnet activity. It is Bittensor's native sybil-resistance: throwaway hotkeys cost real money in an active subnet.

Machine learning

Training recipe. The combination of model architecture, optimizer, learning-rate schedule, data mixture, regularization, and training procedure that together produce a model. Ralph's canonical artifact per track is the best known open recipe for that track's objective.

Pretraining vs. post-training. Pretraining trains a base model from scratch on a broad corpus. Post-training (supervised fine-tuning, preference tuning, RL) adapts a base model to a target task or behavior. Ralph tracks each separately because their objectives, data, and evaluation regimes differ.

Validation perplexity, val_bpb. A standard pretraining metric — lower is better. The original autoresearch project optimizes a closely related quantity (validation bits-per-byte).

MMLU, HellaSwag, ARC. Multiple-choice benchmarks widely used to evaluate language-model knowledge and reasoning. Ralph's hidden-eval set draws from held-out subsets of these (Section 5.7).

FID. Fréchet Inception Distance — the standard quality metric for image-generation models. Native objective of any future diffusion track.

FSDP, ZeRO, Megatron. Distributed-training stacks for large models: Fully-Sharded Data Parallel, ZeRO optimizer state sharding, and Megatron tensor parallelism. Phase-2 infrastructure for Ralph's larger-scale tracks.

Ralph-specific

Track. A competition within Ralph defined by a model class and objective: its own canonical baseline, eval pool, scoring weights, model-size ladder, and miner population. Launches with small LLM pretraining; expands over time (Section 5.10).

Canonical baseline / canonical recipe. A track's public Git repository containing the best-known training recipe for that track's objective at any moment. Every accepted patch in the track is merged here.

Experiment record. The structured, openly-licensed corpus of every submission Ralph has ever processed across every track — hypothesis, patch, conditions, result, attestation — including verified negative results.

Attested execution (proof-test tier). Ralph's single execution requirement for proof-test runs. Every submission runs the official Ralph Docker inside a Confidential VM (Intel TDX or AMD SEV-SNP) on a Confidential-Computing GPU (H100/H200/B200), carrying a nonce-bound attestation chain that binds the GPU report, the CPU quote, the container measurement, the data-manifest hash, and a rolling log hash. Submissions without a valid chain are rejected, not discounted; there is no lower tier.

The 90/10 split. Per-round miner emissions: 90% to the verified winner, 10% across submissions clearing the meaningfulness bar of Section 5.6.

Decisively-beats-the-king margin. The noise-floor score margin a submission must exceed to displace the current canonical baseline, set empirically by running the unchanged baseline many times on the active eval to measure score variance.

Proof test. The canonical training procedure that the official Ralph Docker runs on a miner's CC-capable GPU when the miner wants to submit a candidate patch. Produces a checkpoint, training log, calibration result, and a nonce-bound TDX + nvtrust attestation chain. The proof test is the only training the protocol attests; the miner's earlier search loop is private and unattested.

Proof-test handshake. The on-chain protocol step that initiates a proof-test run. The miner announces a patch hash; the protocol returns a beacon-derived nonce; the miner's Docker uses that nonce as input to every attestation in the resulting run.

Continuous-attestation chain. The sequence of per-epoch TDX + nvtrust attestations produced by the official Docker during a proof-test run, each chained to the previous via a rolling hash of the training log. Validators verify the full chain asynchronously at submission time; no live monitoring is required.

program.md. A miner's editable configuration file that defines their autonomous research organization — the agent's roles, strategy, risk appetite, and which parts of the codebase it may modify. The unit of competition in Ralph's meta-layer (Section 5.9).

Hardware attestation

TEE (Trusted Execution Environment). A hardware-isolated memory region whose contents are cryptographically protected from the host operating system and hypervisor. Modern examples include Intel TDX and AMD SEV-SNP.

Intel TDX. Intel Trust Domain Extensions — the CPU-side TEE used in Ralph's attested execution layer to attest the confidential VM in which a training run executes.

NVIDIA Confidential Computing / nvtrust. Hardware confidential computing on NVIDIA H100/H200/B200 GPUs, with the nvtrust open-source library providing the verification path. Reports GPU state and workload integrity, attested at attestation time (not continuously — see Section 5.4).

Attestation nonce. A cryptographically-strong value the validator issues at run-start, included as input to both the TDX and NVIDIA attestation reports. Its presence in both quotes binds the GPU and CPU attestations to the same submission.

References

All figures attributed to their sources; cost estimates vary by methodology and are reported as ranges where sources disagree. Accessed May 2026.

[1] Cottier, Rahman et al., "The Rising Costs of Training Frontier AI Models," Epoch AI / arXiv 2405.21015. https://arxiv.org/abs/2405.21015

[2] Epoch AI, "How much does it cost to train frontier AI models?" https://epoch.ai/blog/how-much-does-it-cost-to-train-frontier-ai-models

[3] Stanford HAI, "AI Index Report 2025" (training-cost estimates). https://hai.stanford.edu/ai-index

[4] Deluair, "Frontier AI training cost trajectory 2026." https://deluair.com/consultancy/insights/frontier-ai-training-cost-2026

[5] Karpathy, A., "autoresearch" (GitHub repository and README). https://github.com/karpathy/autoresearch

[6] Cortés, D., "Autoresearch isn't just for training models," Shopify Engineering, Apr 2026. https://shopify.engineering/autoresearch

[7] CoinGecko, "Top 5 Bittensor Subnets: A Deep Dive into the dTAO Ecosystem." https://www.coingecko.com/learn/top-bittensor-subnets-dtao

[22] Recursive, "First Steps Toward Automated AI Research" (June 2026) — automated research system reaching SOTA on NanoChat autoresearch, NanoGPT-Speedrun, and NVIDIA SOL-ExecBench; beat the autoresearch@home human-and-agent crowd. https://www.recursive.com/articles/first-steps-toward-automated-ai-research

[23] ScaleAutoResearch (Y. Wang et al.) and Prime Intellect — automated training-recipe search run at ~124M (NanoGPT-Speedrun) scale at low per-run cost; cited for the 2026 collapse in the cost of running the autoresearch loop. https://github.com/recursive-org/first-steps-toward-automated-ai-research

[8] The Defiant, "Bittensor Subnet Tokens Surge as TAO Rally Boosts Ecosystem," Mar 2026. https://thedefiant.io/news/markets/bittensor-subnet-tokens-surge-as-tao-rally-boosts-ecosystem

[9] Epoch AI, "How far can decentralized training over the internet scale?" https://epoch.ai/gradient-updates/how-far-can-decentralized-training-over-the-internet-scale

[10] Prime Intellect, technical reports and Lab platform announcements (INTELLECT-1 / INTELLECT-3). https://www.primeintellect.ai/blog/lab

[11] MIT Technology Review, "OpenAI is throwing everything into building a fully automated researcher," Mar 2026. https://www.technologyreview.com/2026/03/20/1134438/openai-is-throwing-everything-into-building-a-fully-automated-researcher/

[12] Sakana AI et al., "The AI Scientist: Towards Fully Automated Open-Ended Scientific Discovery," arXiv 2408.06292. https://arxiv.org/abs/2408.06292

[13] o-mega, "Self-Improving AI Agents: The 2026 Guide" (verifiability constraint). https://o-mega.ai/articles/self-improving-ai-agents-the-2026-guide

[14] Karpathy × Sarah Guo, podcast on autoresearch and autonomous AI research (transcript provided by project author). https://x.com/saranormous/status/2035080458304987603

[15] tao.media, "The Conviction Upgrade: Bittensor Just Made Subnet Owner Exits a Public Event," May 2026. https://www.tao.media/the-conviction-upgrade-bittensor-just-made-subnet-owner-exits-a-public-event/

[16] Bittensor documentation, "Understanding Subnets" (dTAO, alpha tokens, emissions, AMM pricing). https://docs.learnbittensor.org/subnets/understanding-subnets

[17] Mordor Intelligence, “Machine Learning as a Service (MLaaS) Market” (market size and segmentation, 2026). https://www.mordorintelligence.com/industry-reports/global-machine-learning-as-a-service-mlaas-market

[18] Coverage of Nous Research’s Hermes Agent — open-source self-evolving agent; GitHub-trending and OpenRouter usage data, 2026. https://www.roborhythms.com/hermes-agent-tops-openrouter-may-2026/

[19] Manifold Labs, “Targon (Bittensor SN4): Verifiable AI Inference” — production deployment of NVIDIA Confidential Computing nested in Intel TDX on Bittensor. https://github.com/manifold-inc/targon

[20] NVIDIA, “nvtrust: Ancillary open-source software to support Confidential Computing on NVIDIA GPUs” (H100/H200/B200 attestation). https://github.com/NVIDIA/nvtrust

[21] Independent coverage of Bittensor revenue-funded buybacks at Chutes (SN64), Rayon Labs (CoinGecko, ChainUp, subnetalpha.ai). https://www.coingecko.com/learn/top-bittensor-subnets-dtao

Appendix A. Illustrative Experiment Record

Every Ralph submission carries a structured, timestamped record so that each emission the network pays is traceable to a specific, reproducible, attributed contribution. A simplified example:

```
{
  "idea": "replace AdamW beta2 schedule with dynamic beta2",
  "files_changed": ["optimizer.py"],
  "expected_benefit": "lower loss instability in early training",
  "compute_budget": "8xH100, 4 hours",
  "baseline_score": 0.812,
  "new_score": 0.827,
  "cost": "$92",
  "seeds": [1, 2, 3],
  "stages_passed": [1, 2, 3, 4, 5],
  "audited": true,
  "accepted": true
}
```

In production the record also includes miner identity, base-commit hash, patch hash, full per-seed metrics, throughput and memory measurements, and validator signatures.

Appendix B. Glossary

Ralph sits at the intersection of two technical communities. This glossary covers the terms most likely to be unfamiliar to a reader from either side.

Bittensor and dTAO

Term	Meaning
Alpha token	Each subnet has its own market-priced token, paired with TAO in an automated-market-maker (AMM) liquidity pool. Staking TAO into a subnet mints alpha; its price is the ratio of the pool's TAO to alpha reserves.
Auto-staking buyback	A mechanism in which a subnet's revenue (in TAO or stablecoin-converted) is programmatically used to purchase the subnet's own alpha on its dTAO pool, creating buy pressure tied to product usage rather than speculation. Operated by Rayon Labs on Chutes (SN64).
Bittensor	A decentralized network in which subnets specialize in different machine-intelligence tasks; miners contribute work and validators score it; the native TAO token is emitted in proportion to assessed value.
Coldkey / Hotkey	Two-key model: a coldkey holds and authorizes spending of TAO; a hotkey is the public-facing identity a miner or validator uses on a subnet.
dTAO (Dynamic TAO)	The February 2025 protocol upgrade that gave every subnet its own alpha token and AMM pool, letting the market — not a central authority — allocate TAO emissions across subnets by repricing their alpha.
Emissions	TAO (and per-subnet alpha) that the protocol issues each block to validators, miners, and the subnet owner. The split is determined by Yuma Consensus and per-subnet configuration.
Hidden / rotating eval set	Validator-held evaluation data that the miner never sees, refreshed on a published cadence under commit-reveal randomness so that overfitting to a fixed target is structurally impossible.
Recycle fee	The TAO fee a participant pays to register a hotkey on a subnet. It scales with subnet demand; once paid it is recycled (deducted from total issuance) rather than going to any party.
Subnet	A specialized network within Bittensor focused on one machine-intelligence task. Identified by netuid; capped at 128 active slots, expanding to 256.
Subnet slot / deregistration	Because slots are capped, when a new subnet registers the lowest-supported existing subnet is deregistered — making market support an existential, not optional, concern for any subnet.
TAO	Bittensor's native protocol token. Hard-capped at 21 million; halving schedule cut daily emission from 7,200 to 3,600 TAO in December 2025.
Validator weights	Each validator publishes per-miner weights summarizing the validator's assessment of the miner's value. Yuma Consensus aggregates weights into the final emission allocation.
Yuma Consensus	Bittensor's algorithm for aggregating validator weights into honest, consensus-aligned emission distributions across miners and validators.

Machine learning and training

Term	Meaning
AdamW	A widely used adam-style optimizer with decoupled weight decay; standard for LLM training. Karpathy's autoresearch agent reportedly found improvements to its beta-schedule that a human expert had missed.

Term	Meaning
Attestation (TEE)	A cryptographic signature from a hardware Trusted Execution Environment that proves a specific workload ran inside a specific protected enclave. Ralph uses Intel TDX (CPU) nested with NVIDIA Confidential Computing (GPU).
Calibration benchmark	A small, deterministic compute test (matmul + attention + collective) run on the miner's machine alongside the training run; its timings fingerprint the hardware architecture and let validators verify cost claims.
Canonical baseline	The current best-known open recipe for a track — a Git repository that absorbs every accepted patch and is the starting point for every subsequent miner.
Confidential Computing	NVIDIA's feature on H100/H200/B200 GPUs that runs workloads inside a hardware-protected enclave and emits a signed attestation report. The hardware primitive Ralph relies on for the proof-test attestation chain (Section 5.4); every submission requires it.
FID	Fréchet Inception Distance — the standard quality metric for image generation models; lower is better. Used as the objective on a hypothetical diffusion track.
FSDP / ZeRO	Fully-Sharded Data Parallel / Zero Redundancy Optimizer — distributed-training techniques that shard model state across many GPUs to fit larger models in aggregate memory.
GEPA	Genetic / evolutionary prompt-and-code optimization against benchmarks — the core self-improvement mechanism in Hermes Agent and a likely substrate for Ralph's meta-layer.
KL-divergence	Kullback–Leibler divergence — a measure of how one probability distribution differs from another; used by Distil (SN97) to score how closely a student model's output distribution matches the teacher's.
MMLU	Massive Multitask Language Understanding — a 57-subject benchmark widely used to compare LLM general-knowledge performance.
MoE	Mixture-of-Experts — an architecture in which only a subset of model parameters is active per token, allowing larger total parameter counts at lower per-token compute.
nanochat	Karpathy's small, didactic LLM training stack that autoresearch ships against. Provides the baseline for Ralph's launch track.
Patch	A diff (in the standard Git sense) submitted by a miner against the canonical baseline. Ralph requires patch-only submission so that what a miner changed is auditable and restricted files are protected.
Pretraining vs. post-training	Pretraining: training a base model on large unlabeled corpora. Post-training: subsequent supervised fine-tuning, preference tuning, or reinforcement learning that specializes the base model on tasks like instruction-following or coding.
program.md	In autoresearch and Ralph, a human-editable Markdown file that configures the autonomous research agent: roles, strategy, risk appetite, which parts of the codebase it may touch. Karpathy: "a research organization is a set of program.md files."
Track	A self-contained competition on Ralph: a model class + objective + canonical baseline + eval pool + scoring weights + model-size ladder. The network launches with one track and adds others over time.
val_bpb	Validation bits-per-byte — a normalized form of validation loss; the metric autoresearch uses by default.